

UNIVERSITÀ DEL SALENTO



**Dipartimento di Scienze dell'Economia
Corso di Laurea in Management Digitale**

**Previsione del Fatturato Aziendale:
un'Analisi Comparativa tra l'algoritmo
Prophet e le Reti Neurali Artificiali LSTM**

Laureando:
Francesco Renna

Relatore:
prof. Durante Fabrizio

ANNO ACCADEMICO 2022/2023

Ringrazio tutta la mia famiglia,
in particolare mia madre Pisana, mia sorella Chiara e la mia ragazza Giulia,
le tre donne più importanti della mia vita.
Ma soprattutto vorrei ringraziare i miei gatti, passati e presenti,
a cui sono molto legato
e che mi hanno aiutato a superare momenti difficili della mia vita.

Indice

Elenco delle figure.....	4
Introduzione	5
Capitolo 1 : La Previsione del Fatturato Aziendale e il Suo Ruolo	6
1.1 <i>Il Problema Aziendale</i>	6
1.1.1 <i>Complessità del Contesto Aziendale e l'Importanza della Precisione.....</i>	6
1.2 <i>L'Utilizzo di Metodi Statistici e algoritmi di Machine Learning.....</i>	7
1.2.1 <i>Analisi delle Serie Storiche e la loro Valutazione</i>	7
1.3 <i>Pianificazione Finanziaria.....</i>	8
1.4 <i>Conclusioni</i>	9
Capitolo 2 : L'algoritmo Prophet ed il suo uso in R	10
2.1 <i>Introduzione a R.....</i>	10
2.2 <i>Serie Temporal: Tra Accuratezza e Complessità dei Modelli</i>	11
2.3 <i>Facebook Prophet e la sua Struttura</i>	11
2.4 <i>Valutazione del modello</i>	13
2.5 <i>Considerazioni sul processo di previsione con Prophet ed R</i>	14
Capitolo 3 : Previsione con le Artificial Neural Networks (ANN)	15
3.1 <i>Alcuni cenni storici</i>	15
3.2 <i>Breve introduzione alle Reti Neurali Artificiali</i>	15
3.2.1 <i>Anatomia di una Rete Neurale: Neuroni Biologici ed Artificiali</i>	16
3.3 <i>Classificazione di Reti Neurali Artificiali</i>	17
3.3.1 <i>Algoritmi di apprendimento.....</i>	18
3.3.2 <i>Architettura delle Reti Neurali Artificiali</i>	18
3.3.2.1 <i>Reti ad un solo nodo: i Percettroni.....</i>	18
3.3.2.2 <i>Reti con nodi nascosti: le Reti Feed-Forward</i>	19
3.3.2.2 <i>Reti Ricorrenti: approfondimento sulle Reti LSTM</i>	20
3.4 <i>Settori d'applicazione delle ANN.....</i>	22
Capitolo 4 : Confronto tra Prophet e Reti LSTM.....	24
4.1 <i>Prophet.....</i>	24
4.2 <i>Reti LSTM</i>	31
4.2.1 <i>Breve Introduzione a Pyhton e Analisi</i>	31
4.3 <i>Motivazioni della scelta di diversi Dataframe</i>	37
Conclusioni	43
Bibliografia	44

Elenco delle figure

Figure 3.1: Anatomia di un neurone	16
Figure 3.2: Struttura di una ANN con uno strato nascosto ed un output	17
Figure 3.3: Struttura di un perceptrone.....	19
Figure 3.4: Struttura di una rete feed-forward	20
Figure 3.5: Confronto rete RNN classica e LSTM	21
Figure 3.6: Anatomia rete LSTM.....	22
Figure 4.1: Andamento del Fatturato su R.....	25
Figure 4.2: Andamento delle previsioni effettuate e Changepoints.....	28
Figure 4.3: Componenti principali della previsione	29
Figure 4.4: Scatter Plot su R	30
Figure 4.5: Andamento del Fatturato su Python	32
Figure 4.6: Confronto tra le previsioni e i dati reali.....	35
Figure 4.7: Scatter plot su Python.....	36
Figure 4.8: Andamento Previsioni e Scatter Plot su R.....	39
Figure 4.9: Andamento Previsioni e Scatter Plot su Python	41

Introduzione

La capacità di anticipare le dinamiche finanziarie è diventata essenziale per una gestione aziendale efficace a causa della crescente complessità del panorama d'impresa.

L'obiettivo della presente tesi è esaminare e confrontare due tecniche avanzate per la previsione del fatturato aziendale. Il primo metodo è caratterizzato dall'utilizzo dell'algoritmo Prophet su R, mentre come secondo approccio si utilizzerà Python con le Reti Neurali Artificiali, in particolare le reti LSTM (*Long Short-Term Memory*).

Prophet, sviluppato da Facebook, è un potente strumento progettato per modellare serie temporali caratterizzate da stagionalità e pattern complessi. Dall'altro lato, l'utilizzo delle reti LSTM è emerso come un approccio avanzato e flessibile per la modellazione di serie temporali più complesse.

L'elaborato esplorerà approfonditamente entrambi i metodi, analizzando le loro caratteristiche, il processo di implementazione e confrontando le prestazioni di entrambi su dati reali.

Infine, attraverso questo studio, si mira a un quadro esaustivo per la scelta del metodo più adatto a esigenze specifiche, promuovendo una migliore comprensione delle potenzialità di Prophet e delle reti LSTM nella previsione del fatturato aziendale.

Capitolo 1 : La Previsione del Fatturato Aziendale e il Suo Ruolo

Le aziende sono costantemente alla ricerca di metodi e strumenti per ottenere un vantaggio rispetto ai loro concorrenti nel mondo frenetico e competitivo degli affari. La previsione del fatturato è uno degli elementi più importanti che influenzano il successo di un'azienda. La capacità di prevedere con precisione le entrate future non solo fornisce una guida strategica, ma può anche aiutare l'azienda a prendere decisioni operative intelligenti, ottimizzare le risorse e aumentare la redditività complessiva.

Gli esperti utilizzano una vasta gamma di tecniche e strumenti di previsione nelle loro analisi, coprendo così una vasta area nel campo delle scienze informatiche. Tra i software più affidabili e ampiamente utilizzati spiccano R e Python. Entrambi sono tra i principali linguaggi di programmazione utilizzati in tutto il mondo per molteplici scopi, come strumenti dedicati sia all'analisi statistica sia all'implementazione di algoritmi di Machine Learning.

In questo capitolo, si analizzerà la questione della previsione del fatturato e come gli strumenti informatici siano importanti per il successo di un'azienda.

1.1 Il Problema Aziendale

La previsione del fatturato aziendale è una sfida complessa che richiede di determinare le entrate finanziarie ed economiche future di un'azienda per un determinato orizzonte temporale. Il processo richiede l'analisi di una varietà di dati, tra cui le condizioni di mercato, le tendenze storiche delle vendite, le fluttuazioni economiche, i cambiamenti nel comportamento dei clienti e molti altri. Inoltre, il contesto in cui opera un'azienda può variare notevolmente a seconda del settore, della dimensione e della sede. Di conseguenza, la previsione del fatturato non dovrebbe essere sottovalutata: è una sfida importante che richiede un approccio ben organizzato e l'uso di strumenti sofisticati.

1.1.1 Complessità del Contesto Aziendale e l'Importanza della Precisione

Il contesto aziendale è estremamente complesso e delicato dal momento che vi sono molte variabili che possono influenzare il fatturato di un'azienda. Ad esempio, chi lavora a stretto contatto con i clienti come le aziende manifatturiere devono tener conto della domanda dei clienti stessi, della disponibilità di materie prime, della capacità produttiva e della concorrenza sul mercato. D'altra parte, fattori come le variazioni delle tariffe di mercato e le richieste stagionali dei clienti possono avere un impatto su un'azienda di servizi. Di conseguenza una precisa ed accurata previsione del fatturato deve essere personalizzata per questa diversità di influenze.

La previsione del fatturato è importante per un'azienda. Deve essere accurata per consentire all'azienda di pianificare gli obiettivi con certezza e di affrontare i problemi in modo efficace che potrebbero sorgere. L'azienda potrebbe trovarsi in situazioni di sovrapproduzione o sovra-investimento, con conseguente spreco di

risorse, se le previsioni fossero troppo ottimistiche. Al contrario, previsioni troppo conservative potrebbero causare perdite di reddito e opportunità di vendita. Inoltre, la mancanza di precisione nella previsione può avere un impatto negativo sulla fiducia degli investitori e dei finanziatori, con eventuali danni per l'azienda.

1.2 L'Utilizzo di Metodi Statistici e algoritmi di Machine Learning

Sempre più aziende stanno utilizzando tecniche di machine learning, in italiano “apprendimento automatico”, e metodi statistici per affrontare la complessità della previsione del fatturato. Queste tecniche consentono l'analisi di una grande quantità di dati, la scoperta di modelli nascosti e la creazione di previsioni più accurate, che contribuiscono significativamente alla strategia aziendale e all'efficacia operativa. Grazie alla loro versatilità e alla vasta gamma di librerie disponibili, l'utilizzo di software come R e Python si è rivelato essenziale per implementare questi metodi.

In particolare, il machine learning è un sottosettore di intelligenza artificiale che si concentra sull'utilizzo di dati e algoritmi per imitare il modo in cui gli esseri umani apprendono e migliorano progressivamente la loro accuratezza [5].

Gli algoritmi di machine learning sono utilizzati per la classificazione, la previsione e la scoperta di informazioni chiave. La classificazione comporta la categorizzazione dei dati in gruppi basati su determinate caratteristiche, come lo spam o le transazioni fraudolente. I modelli predittivi possono essere sviluppati utilizzando l'analisi dei dati storici, come nel settore finanziario, per prevedere le fluttuazioni del mercato azionario o il comportamento dei clienti [8].

Le intuizioni ottenute dal machine learning possono aiutare le aziende a prendere decisioni più informate e a sviluppare strategie aziendali efficaci. L'apprendimento automatico aiuta anche ad affrontare la sfida dei big data, in quanto automatizza l'analisi dei dati, identificando tendenze, anomalie e modelli in modo efficiente.

Infine, il machine learning è una componente essenziale della scienza dei dati, fornendo strumenti per la classificazione, la previsione, la scoperta di informazioni e la gestione dei big data. Mentre la disponibilità dei dati si espande e le aziende diventano più consapevoli del potenziale dei loro dati, si prevede che la domanda di scienziati ed esperti di dati nella combinazione di scienza dei dati e machine learning crescerà in modo significativo.

1.2.1 Analisi delle Serie Storiche e la loro Valutazione

L'analisi delle serie storiche è il metodo più comune per prevedere il fatturato. Questa tecnica utilizza dati storici delle vendite per scoprire cicli, tendenze e stagionalità che possono influenzare il comportamento futuro delle entrate aziendali. L'ARIMA (Autoregressive Integrated Moving Average) e il SARIMA (Seasonal ARIMA) sono algoritmi di analisi delle serie storiche che possono essere utilizzati in R e Python per produrre previsioni basate su modelli temporali. Quando le tendenze o le stagioni influenzano le vendite, questi modelli sono molto utili.

L'analisi predittiva è un tipo di analisi aziendale che utilizza l'apprendimento automatico per creare un modello predittivo per applicazioni specifiche dell'azienda. Di conseguenza, include una serie di metodi statistici, compresa la modellazione predittiva e l'apprendimento automatico, che utilizzano l'analisi dei fatti storici e attuali per fare previsioni su eventi futuri o altrimenti sconosciuti. Rappresenta un gruppo significativo di applicazioni dell'apprendimento automatico [26].

I modelli predittivi negli affari identificano opportunità e rischi utilizzando dati storici e transazionali. I modelli registrano le connessioni tra diversi fattori per consentire la valutazione del rischio o del potenziale correlato an un determinato set di circostanze [3].

L'efficacia dei modelli di previsione deve essere valutata attentamente attraverso metodi appropriati. L'uso di cross-validation e l'analisi delle metriche di errore, come l'errore quadratico medio (RMSE) o il coefficiente di determinazione (R-quadro), aiutano a determinare la bontà dei modelli e a identificare aree in cui possono essere migliorati. L'iterazione continua è fondamentale per raffinare i modelli e mantenerli adeguati alle esigenze aziendali in evoluzione.

La previsione del fatturato aziendale non si limita a calcoli statistici e algoritmi sofisticati. Anche il supporto decisionale è un problema. Per trasformare le previsioni in fatti concreti, sono necessari strumenti di supporto decisionale basati sui dati. Queste risorse consentono alle aziende di valutare le conseguenze finanziarie di una varietà di piani e di prendere decisioni informate.

In sintesi, le aziende possono utilizzare un potente strumento per migliorare la previsione del fatturato utilizzando metodi statistici e di apprendimento automatico supportati da software come R e Python. Queste tecniche consentono di gestire i dati aziendali complessi e di adattarsi rapidamente alle dinamiche di mercato che cambiano continuamente. Tuttavia, è importante sottolineare che la previsione del fatturato è un processo continuo che richiede attenzione, aggiornamenti e miglioramenti per mantenere l'azienda competitiva nel mercato globale.

1.3 Pianificazione Finanziaria

La previsione finanziaria rappresenta un elemento cruciale per il successo a lungo termine di qualsiasi azienda. Essenzialmente, si tratta di guardare nel futuro finanziario dell'azienda, analizzando dettagliatamente le entrate previste e i costi associati. Ampliare questa prospettiva può chiarire ulteriormente quanto sia fondamentale questo processo:

- Ottimizzazione dei risultati: una previsione accurata del fatturato consente di assegnare in modo efficace i fondi per sostenere le spese necessarie e progetti di investimento programmati.
- Riduzione del livello di rischio: un'azienda può affrontare meglio le sfide, come fluttuazioni del mercato o imprevisti economici, quando ha una chiara visione delle sue entrate e uscite future.

- **Pianificazione Strategica:** queste previsioni sono utili per le aziende per trovare opportunità di crescita, determinare il momento giusto per diversificare o espandersi e stabilire obiettivi finanziari realistici.
- **Comunicazione con Investitori e Stakeholder:** gli investitori e i partner commerciali richiedono frequentemente previsioni finanziarie; quindi, per aumentare la fiducia degli investitori e migliorare le relazioni con gli stakeholder, le previsioni dovrebbero essere accurate il più possibile.
- **Valutazione delle Prestazioni:** le previsioni finanziarie possono essere utilizzate come base per valutare le prestazioni di un'azienda. L'organizzazione può identificare le variazioni significative confrontando i risultati reali con le previsioni e applicare le modifiche necessarie.
- **Pianificazione Fiscale:** consente di calcolare con precisione le imposte dovute e di adottare strategie fiscali efficaci per ottimizzare la sua posizione fiscale.

In sintesi, la previsione finanziaria è molto più di un'attività contabile. È un processo strategico che informa le decisioni aziendali, consente la gestione efficiente delle risorse e contribuisce alla sostenibilità e alla crescita dell'azienda a lungo termine. L'uso di software per la pianificazione finanziaria può semplificare notevolmente questo processo, fornendo strumenti sofisticati per analizzare e ottimizzare la situazione finanziaria dell'azienda.

1.4 Conclusioni

La previsione del fatturato aziendale è una sfida critica per qualsiasi azienda. La capacità di prevedere con precisione le entrate future è essenziale per la pianificazione strategica, la gestione finanziaria e il successo complessivo. L'uso di metodi statistici e di machine learning, supportati da software come R e Python, ha reso possibile migliorare notevolmente l'accuratezza delle previsioni. Tuttavia, è importante ricordare che la previsione del fatturato è solo il primo passo, la traduzione di queste previsioni in azioni concrete attraverso strumenti di supporto decisionale è ciò che permette alle aziende di ottenere un vantaggio competitivo nel mercato sempre mutevole degli affari.

Nel prossimo capitolo, verranno esplorate alcune delle sfide e delle considerazioni pratiche legate all'implementazione di sistemi di previsione del fatturato aziendale.

Capitolo 2 : L'algoritmo Prophet ed il suo uso in R

Nelle aziende moderne, la capacità di prevedere con precisione il proprio stato economico/finanziario è essenziale per il successo e la pianificazione strategica. In questo capitolo si analizzerà un potente mezzo per affrontare questo problema: il linguaggio di programmazione R e soprattutto la libreria "*Prophet*", creata da Facebook.

R consente la manipolazione, l'analisi e la visualizzazione efficace dei dati, costituendo una risorsa essenziale per gli analisti di dati e gli esperti di previsione.

La libreria "*Prophet*" gestisce le previsioni di serie storiche, rendendo il processo di previsione del fatturato dell'azienda più semplice ma allo stesso tempo preciso ed affidabile.

Verranno esaminati in dettaglio le metodologie di utilizzo di R e "*Prophet*" per analizzare i dati storici dell'azienda, trovare tendenze e stagionalità e creare modelli di previsione accurati.

2.1 Introduzione a R

Il software è stato sviluppato inizialmente da Ross Ihaka e Robert Gentleman nel 1993, mentre ora è supportato da R Core e R Foundation for Statistical Computing. Grazie alla sua natura open-source, R ha visto una crescente adozione e sviluppo da parte di una vasta comunità di utenti e sviluppatori in tutto il mondo.

Una delle caratteristiche distintive è la vasta collezione di pacchetti di estensione. Questi pacchetti, creati dai membri della comunità, forniscono funzionalità aggiuntive, metodi statistici avanzati e strumenti di visualizzazione dei dati che arricchiscono notevolmente il linguaggio di base. La disponibilità di questa vasta libreria di pacchetti facilita l'analisi dei dati e lo sviluppo di software statistico personalizzato [11].

R è ampiamente utilizzato in molte discipline, tra cui la statistica, l'analisi dei dati, la bioinformatica, l'economia e molte altre. La sua popolarità è testimoniata da numerosi sondaggi tra gli utenti e da studi accademici che lo posizionano come uno dei linguaggi di programmazione più utilizzati nell'estrazione e nell'analisi dei dati. Tuttavia, è importante notare che la sua posizione nell'indice TIOBE, che misura la popolarità dei linguaggi di programmazione, può variare nel tempo, ma la sua presenza costante tra i primi posti dimostra la sua rilevanza nel campo della programmazione statistica [10].

Oltre all'analisi dei dati, R è ampiamente utilizzato per creare grafici e visualizzazioni coinvolgenti. Grazie alla sua capacità di generare grafici di alta qualità e interattivi, R è uno strumento essenziale per comunicare i risultati delle analisi in modo efficace e comprensibile.

In sintesi, R continua a essere un linguaggio di programmazione fondamentale nel mondo della statistica e dell'analisi dei dati. La sua comunità attiva e la vasta gamma di pacchetti di estensione rendono R una scelta eccellente per chiunque sia coinvolto nell'elaborazione e nell'interpretazione dei dati.

2.2 Serie Temporal: Tra Accuratezza e Complessità dei Modelli

Lavorare con dati in serie temporali può essere difficile a causa dell'aleatorietà dei fenomeni, e i diversi algoritmi utilizzati per generare i modelli possono essere molto sofisticati e difficili da regolare. Questo è particolarmente vero quando si lavora con dati più stagionali. Inoltre, i modelli di serie temporali convenzionali come Auto-Regressive integrated Moving Average (ARIMA) oppure Long Short-Term Memory (LSTM) hanno molti requisiti di dati stringenti, come valori equidistanti e stazionarietà [1].

Se non si dispone di una comprensione significativa dell'architettura delle reti neurali, modelli di serie temporali aggiuntivi come le reti neurali ricorrenti con memoria a lungo termine (RNN-LSTM) possono rivelarsi estremamente complessi e difficili da utilizzare. Quindi, l'analisi delle serie temporali è un grande ostacolo per il tipico analista di dati.

2.3 Facebook Prophet e la sua Struttura

All'incirca nel 2017, alcuni ingegneri di Facebook (attuale Meta) hanno pubblicato un report chiamato "Forecasting at Scale", dove all'interno è stato introdotto il progetto open source Prophet, fornendo modelli di serie temporali veloci, potenti e accessibili anche ad analisti non troppo esperti nella previsione.

Lo strumento è utilizzabile in software come Python e R ed è progettato principalmente per affrontare problemi di previsione di dati che presentano stagionalità, tendenze e addirittura festività. Prophet è anche noto per la sua facilità d'uso e la sua capacità di gestire dati mancanti e outliers in modo robusto [24].

Non tutti i problemi di previsione possono essere risolti con la stessa procedura, per questo Prophet è ottimizzato per le attività di previsione aziendale, ma più in generale per serie di dati che presentano alcune delle seguenti caratteristiche:

- osservazioni orarie, giornaliere o settimanali collezionate su un periodo di almeno un anno;
- forti stagionalità multiple "a misura d'uomo": giorno della settimana e periodo dell'anno;
- festività importanti che si verificano a intervalli irregolari noti in anticipo;
- eventuali osservazioni mancanti o di grandi valori anomali, seppure non in numero eccessivo;
- cambiamenti delle tendenze storiche, ad esempio dovuti al lancio di prodotti o alle modifiche alla registrazione;
- trend crescenti e non lineari, con eventuale limite naturale livello di saturazione.

Una delle principali motivazioni dietro la creazione di Prophet risiede nel desiderio di sviluppare previsioni di alta qualità più facili da usare e soprattutto più precise [13].

Prophet può essere sintetizzato con un semplice modello di regressione $y(t)$ e tre principali componenti: trend, stagionalità e festività. L'equazione può essere scritta come:

$$y(t) = g(t) + s(t) + h(t) + \varepsilon(t), \quad t \geq 0,$$

dove $g(t)$ descrive la componente di trend, $s(t)$ invece rappresenta la stagionalità mentre $h(t)$ le festività nazionali, infine $\varepsilon(t)$ è l'errore sistematico.

In breve, i tre modelli principali sono i seguenti. [1]

- Trend $g(t)$: l'andamento generale dei dati è modellato dalla funzione di crescita e che può essere presente in tutti i punti dei dati o modificata in quelli che Prophet chiama "punti di cambiamento", punti nei quali i dati cambiano direzione. Ad esempio, la forte diminuzione di nuovi casi di COVID-19 durante la pandemia dopo l'introduzione di un vaccino. I punti di cambiamento sono rilevati automaticamente o possono essere impostati manualmente. La funzione di crescita ha tre opzioni:
 1. *Crescita lineare*: l'impostazione predefinita di Prophet, utilizza un insieme di equazioni lineari ma a tratti con pendenze diverse tra i punti di cambio. Il termine di crescita è simile alla classica equazione $y = mx + b$, tranne per pendenza (m) e intercetta (b) che cambieranno valore ad ogni punto di cambio.
 2. *Crescita logistica*: è utile quando le serie temporali sono limitate o hanno un valore minimo in cui i valori che da modellare diventano saturi. Quando viene utilizzata, il termine di crescita sarà simile ad un'equazione per curva logistica, tranne per il fatto che la capacità di carico (C) varierà in funzione del tempo, il tasso di crescita (k) e l'offset (m) sono variabili, mentre (x) rappresenta la misura della crescita rispetto alla capacità massima nel contesto della curva. Quando è vicina a 0, la crescita è lenta, quando si avvicina a 1, la capacità di carico è quasi completamente saturata:

$$g(t) = \frac{C(t)}{1 - x^{-k(t-m)}}$$

3. *Piatto*: non vi è un trend di crescita, ma è possibile comunque la presenza di stagionalità.

- Stagionalità $s(t)$: aiuta il modello a considerare quei cambiamenti periodici in base al ritmo giornaliero, settimanale, mensile e annuale. Per adattare e prevedere gli effetti della stagionalità, Prophet si affida alla serie di Fourier per fornire un modello flessibile. L'approssimazione per la stagionalità può essere riassunta in:

$$s(t) = \sum_{n=1}^N \left(a_n \cos\left(\frac{2\pi nt}{P}\right) + b_n \sin\left(\frac{2\pi nt}{P}\right) \right)$$

Dove P è il regolare periodo della serie storica, quindi $P = 365$ per dati annuali, mentre $P = 7$ dati divisi per settimana. I parametri $a_n \dots b_n$ sono stimati secondo $N > 0$, il quale definisce se cambiamenti frequenti possono essere considerati nel modello predittivo per inserirli nei pattern stagionali, oppure se invece questi sono dei semplici errori o outliers.

- Festività $h(t)$: alterano le previsioni in modo inaspettato, come la Diwali, festa induista durante la quale le persone decorano le loro case e i loro luoghi di culto con candele di argilla. Festività che potrebbe, ad esempio, aumentare l'inquinamento atmosferico a causa degli incendi di petardi.

2.4 Valutazione del modello

Vari coefficienti vengono usati per valutare le performance e affidabilità del modello; infatti, tra i quali ritroviamo il coefficiente di determinazione R^2 , $MAPE$ che rappresenta l'errore percentuale assoluto medio, l'errore relativo RE e l'errore quadratico medio $RMSE$:

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{(y_i - y_{mean})^2}$$

$$MAPE = \frac{1}{N} \sum_{i=1}^N \frac{|y_i - \hat{y}_i|}{y_i} \times 100\%$$

$$RE = \frac{|y_t - \hat{y}_t|}{y_t} \times 100\%$$

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

Si sottolinea che la valutazione sarà condotta esclusivamente sul test set [15], ossia sui dati validi esclusi dalla stima del modello.

Qui y_i rappresentano i valori storici, mentre $\hat{y}_i$ sono i dati predetti all'istante i -esimo, y_{mean} è la media dei dati storici, y_t e $\hat{y}_t$ sono rispettivamente il valore storico finale e quello predetto. N è il numero di valori previsti nel periodo stimato.

Più il risultato di R^2 si avvicina a uno, migliore è l'idoneità dei valori previsti, mentre più bassi sono i valori di $MAPE$, RE , $RMSE$, più affidabile sarà il modello.

2.5 Considerazioni sul processo di previsione con Prophet ed R

Prophet si distingue per la sua facilità d'uso e per la sua capacità di produrre previsioni significative ed accurate, anche per coloro che non sono esperti nell'analisi e nelle previsioni di serie temporali. Infatti, come detto in precedenza, questo modello di regressione prende in considerazione tre elementi principali: trend, stagionalità e festività. Questo rende più facile analizzare dati con caratteristiche complesse [21].

La semplicità è una caratteristica importante di Prophet e con l'utilizzo di R, che è noto per le sue capacità di calcolo statistico e visualizzazione dei dati, consente agli utenti di sfruttare al massimo la potenza di Prophet.

Capitolo 3 : Previsione con le Artificial Neural Networks (ANN)

Dopo aver esaminato l'utilizzo della libreria Prophet nel capitolo precedente, ora verrà mostrato un approccio innovativo all'analisi di previsione del fatturato aziendale, concentrandosi sull'applicazione delle Reti Neurali Artificiali (ANN).

Le reti neurali artificiali sfruttano la capacità di apprendimento automatico di modelli altamente adattabili, offrendo un metodo flessibile e potente per affrontare problemi di previsione complessi. I fondamenti delle ANN, inclusi i loro design e metodi di addestramento, verranno presentati in questo capitolo.

3.1 Alcuni cenni storici

Le più antiche teorie sulla funzione cerebrale e sui processi mentali furono concepite dai filosofi Platone e Aristotele. Le prime realizzazioni dei sistemi neurali basati su computer cominciarono nel 1940 con la nascita della cibernetica [7].

Riconoscere caratteri grafici o vocali può essere semplice per gli esseri umani, soprattutto grazie alle conoscenze passate; ma è evidente che risulta particolarmente difficile sviluppare algoritmi analitici capaci di risolvere questi problemi. L'idea di affrontare questo tipo di problemi imitando meccanismi biologici si basa sulla crescente conoscenza della dinamica biologica.

Il libro di Donald Olding Hebb *“L'organizzazione del comportamento”* ha cercato di spiegare i complessi modelli di apprendimento neurale. A lui si deve l'apprendimento *hebbiano*, uno dei fondamentali algoritmi di apprendimento che si basa sul principio secondo il quale se due neuroni si attivano contemporaneamente, allora la loro interconnessione deve essere rafforzata.

Von Neumann nel 1958, tramite il suo libro, ha confrontato il cervello umano con la tecnologia di elaboratori, osservando come le reti neurali non possano essere precise in operazioni complesse a causa della mancanza di struttura [27].

Il concetto di *“Perceptron”* di Frank Rosenblatt fu sviluppato nel 1969, e permise la correzione delle funzioni limitate nella rete neurale [22].

Nel corso degli anni, i ricercatori hanno continuato ad esplorare le limitazioni delle funzioni linearmente separabili, tra cui l'algoritmo XOR, che ha dato nuovo slancio allo studio di questo tipo di reti neurali.

3.2 Breve introduzione alle Reti Neurali Artificiali

Le Neural Networks Artificiali (ANN) sono metodi di previsione basati su modelli matematici che cercano di replicare i neuroni biologici del cervello [29].

Essi rappresentano un sottoinsieme delle tecniche di machine learning e riproducono l'elemento centrale degli algoritmi di deep learning.

Ciascun neurone artificiale è collegato ad un altro e ha una soglia di input che può sopportare e un peso, cioè un parametro che influisce sull'importanza della risposta in uscita. Se l'output di un singolo nodo supera il valore limite, il nodo viene attivato e trasmette i dati al livello successivo della rete. In caso contrario, i dati non vengono trasmessi al livello successivo della rete.

3.2.1 Anatomia di una Rete Neurale: Neuroni Biologici ed Artificiali

Le reti neurali artificiali sono state progettate al fine di imitare, per quanto possibile, il funzionamento di quelle biologiche [20].

L'elemento fondamentale di una rete neurale biologica è il neurone, il quale è formato da tre elementi fondamentali:

- *soma*: corpo centrale del neurone;
- *dendriti*: fibre che si ramificano a partire dal soma e passano attraverso le sinapsi, che servono a raccogliere gli input e trasmetterli al soma;
- *assone*: fibra principale che porta l'output ad altri neuroni.

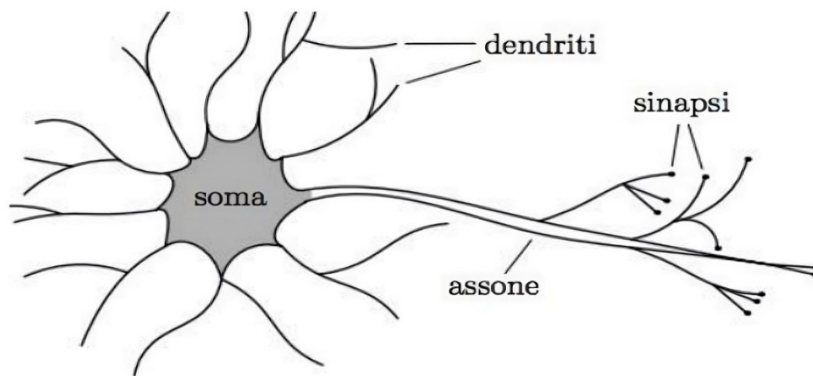


Figure 3.1: Anatomia di un neurone. Si veda [6]

Quando un impulso attraversa la dendriti, il cervello biologico accumula cariche elettriche e funge da condensatore. Ma la soma ha un limite di accumulo, quindi entra nell'assone una volta raggiunta la sua energia massima. Il corpo cellulare trasferisce questa energia alla dendriti di un altro neurone, formando sinapsi e poiché le sinapsi possono verificarsi in più aree contemporaneamente, questo processo consente al cervello di generare una quantità enorme di informazioni.

Al contrario, i computer elaborano i dati in sequenza, completando le attività e passando alla successiva. Ciò ha portato alla possibilità di imitare il sistema biologico attraverso la creazione di reti artificiali.

Esse sono costituite da nodi o neuroni formali, che fungono da unità computazionali fondamentali. Esse sono collegate tra loro per formare un sistema, che include almeno uno strato di ingresso e uno strato di uscita. Anche questa rete

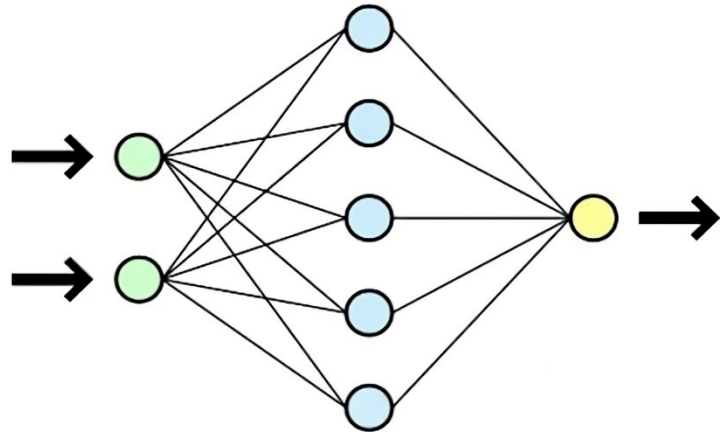


Figure 3.2: Struttura di una ANN con uno strato nascosto ed un output. [20]

ha come obiettivo quello di elaborare dati e informazioni e funziona con un input, simile all'impulso elettrico per il neurone biologico, che arriva al nodo dello strato d'ingresso.

A questo punto, l'input attiva il neurone artificiale e riceve la quantità massima di "stimolo". L'informazione ricevuta può essere trasmessa allo strato di output superando la soglia di ogni nodo, ma un po' come accade tra il corpo cellulare e l'assone, quando l'energia superata viene scaricata essa finirà su un altro neurone.

La differenza qui sta nel fatto che la trasmissione è sotto forma di numeri che vengono trasmessi attraverso funzioni matematiche, invece che di impulsi elettrici.

Affinchè vi sia un'elaborazione di dati precisa e affidabile, vengono studiati vari metodi di Machine Learning. Queste tecniche sono specificamente progettate per addestrare e far imparare la rete neurale.

3.3 Classificazione di Reti Neurali Artificiali

I criteri più utilizzati per raggruppare le ANN sono per tipo di algoritmo di apprendimento e il tipo di struttura.

Le tipologie di algoritmo di apprendimento più famose sono: supervisionato, non supervisionato o per rinforzo.

Mentre le strutture di ANN principali si dividono in: perceptron, reti feed-forward e reti ricorrenti.

3.3.1 Algoritmi di apprendimento

Nell'intelligenza artificiale (AI) e nel machine learning, gli algoritmi di apprendimento sono procedure matematiche utilizzate per far apprendere i computer da dati ed esperienze. Con l'aiuto dell'esperienza, questi algoritmi consentono ai sistemi di migliorare le prestazioni.

L'apprendimento supervisionato è una sottocategoria del machine learning e consiste in un'analisi dei dati che utilizza algoritmi che imparano iterativamente dai dati presi in studio per consentire ai computer di trovare informazioni utili nascoste senza essere esplicitamente programmati su dove cercare. La tecnica di *back-propagation* è una delle molte tecniche utilizzate per migliorare le prestazioni della rete neurale durante la fase di apprendimento. Questa strategia utilizza i dati per modificare i pesi e altri parametri della rete per ridurre l'errore di previsione. In generale questa tecnica di apprendimento permette di "imparare" dai dati storici e di addestramento e applicarli a input sconosciuti per ricavare l'output corretto [16].

L'apprendimento non supervisionato utilizza solo un set di dati senza alcuna indicazione del risultato previsto. Quindi, l'algoritmo cercherà di classificare i dati in base alle caratteristiche comuni per cercare di fare ragionamenti e previsioni sugli input successivi. L'obiettivo di questo metodo è quello di individuare schemi e modelli nascosti, ovvero trovare una struttura logica negli input senza che siano etichettati in anticipo.

3.3.2 Architettura delle Reti Neurali Artificiali

Un'ulteriore classificazione delle reti neurali può essere fatta in base al tipo di architettura, cioè in base al modo in cui sono collegati tra loro i neuroni.

3.3.2.1 Reti ad un solo nodo: i Percettroni

I percettroni sono le reti più elementari e non hanno strati nascosti. Questa rete è composta principalmente da un solo nodo che ottiene input, studia il segnale tramite la funzione e poi restituisce l'output [22].

Il percettrone è un tipo di classificatore binario che mappa i suoi ingressi input x (un vettore di tipo reale) in un valore di output $f(x)$ (uno scalare di tipo reale) calcolato con:

$$f(x) = \sum_{j=1}^N w_j x_j + b_k$$

dove N è il numero input in entrata ricevuti, mentre b_k è il bias, cioè un settaggio della funzione di attivazione e infine w che rappresenta il vettore dei pesi.

Questa rete ha solo uno strato d'ingresso e uno d'uscita; quindi, i dati si muovono in un solo senso, entrando in un punto e uscendo da un altro.

Inoltre, a causa della sua semplicità dovuta alla scarsa quantità di connessioni, l'architettura è molto limitata; quindi, combina i dati in ingresso una sola volta.

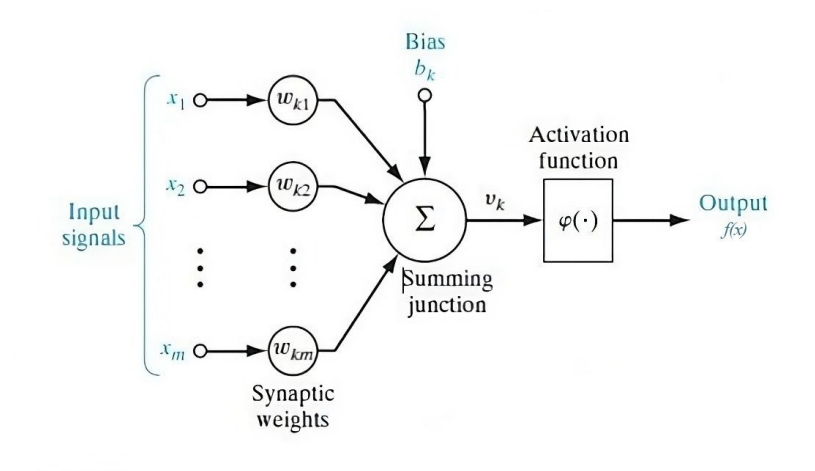


Figure 3.3: Struttura di un percettore. Si veda [6]

Le reti neurali artificiali possono essere utilizzate per risolvere una varietà di problemi, grazie soprattutto al fatto che la struttura fondamentale delle reti neurali artificiali è quasi identica per tutti i sistemi possibili. Questo consente l'utilizzo di teorie e algoritmi identici in macchine utilizzate per diversi scopi. Di conseguenza, è possibile sfruttare dei moduli, cioè blocchi di neuroni che sono in grado di svolgere determinate funzioni, collegandoli ad altri per svolgere attività più complesse rispetto a quelle iniziali.

3.3.2.2 Reti con nodi nascosti: le Reti Feed-Forward

Nel momento in cui vengono aggiunti dei livelli intermedi la rete diventa non lineare. Questa è nota come Rete neurale feed-Forward.

In questo tipo di rete neurale, le connessioni delle unità viaggiano in un unico percorso diretto, seguendo però in maniera simile la logica del percettore.

Con la sua facilità di comprensione e configurazione, una Rete neurale feed-Forward è probabilmente il tipo di rete più utilizzato, infatti la computer science e il data mining in particolare richiedono un comportamento predittivo utilizzando questi tipi di reti neurali.

Le informazioni in una rete neurale feed-Forward entrano nello strato di input, viaggiano attraverso lo strato nascosto ed emergono dallo strato esterno della rete, fornendo all'utente finale una risposta alla sua domanda.

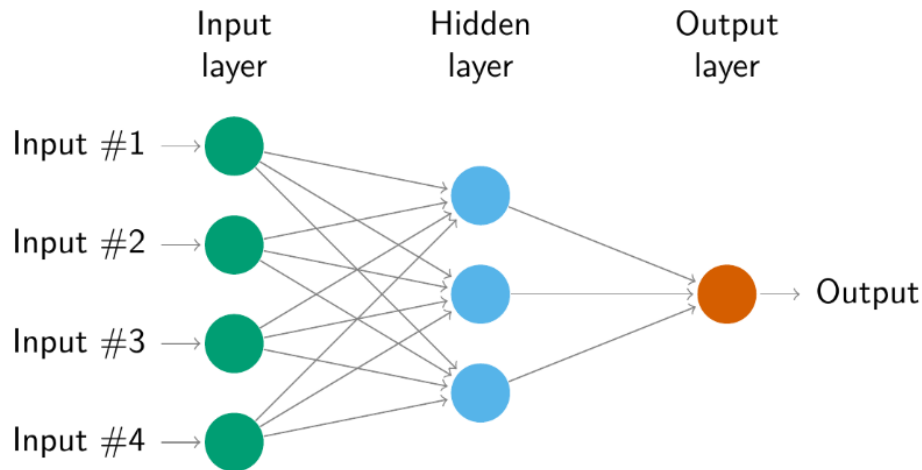


Figure 3.4. Struttura di una rete feed-forward [29]

Un livello di input è semplicemente il posto in cui l'utente inserisce i dati grezzi o i parametri delle informazioni, mentre le fasi centrali della transazione si verificano nello strato nascosto, dove il computer utilizza la sua "esperienza" di gestione di dati simili per generare una risposta presumibilmente soddisfacente.

Il livello di output riceve una risposta dopo che le informazioni sono state incanalate, ma il risultato viene modificato da una funzione non-lineare prima di essere considerata una risposta valida.

I livelli di input di ogni singolo neurone sono combinati secondo la sommatoria v_j vista in precedenza, ma quando si passa allo strato nascosto, si aggiunge la nuova funzione di attivazione che in questo caso è una sigmoidea:

$$s(z) = \frac{1}{1 + e^{-z}}.$$

Ciò tende a ridurre l'effetto dei valori di input massimi, facendo così diventare la rete più robusta ad eventuali outliers [29].

I valori b_1, b_2, b_3 e $w_{1,1} \dots w_{4,3}$ sono "appresi" (o stimati) dai dati presi in osservazione e per evitare che i pesi diventino troppo grandi sono spesso limitati, inizialmente vengono utilizzati valori casuali, ma successivamente vengono modificati utilizzando i dati osservati; infatti, le previsioni di una rete neurale contengono una certa dose di imprevedibilità.

3.3.2.2 Reti Ricorrenti: approfondimento sulle Reti LSTM

Le reti neurali ricorrenti (RNN) sono composte da neuroni che sono collegati tra loro in un ciclo, i quali valori di ingresso di uno strato di livello inferiore vengono utilizzati dai valori di uscita di uno strato superiore [14].

Questa interconnessione tra strati consente ad uno degli strati di essere utilizzato come memoria di stato. Inoltre, fornendo in ingresso una sequenza di valori, consente allo strato di modellare un comportamento dinamico temporale che si basa sulle informazioni acquisite negli istanti di tempo precedenti. In altre situazioni, lo strato è costituito da un gruppo di neuroni con molti loop di connessione sparse che generano una dinamica caotica che viene utilizzata per l'addestramento di un altro segmento della rete. Ciò le rende adatte a compiti che richiedono un'analisi predittiva su sequenze di dati, quali possono essere ad esempio il riconoscimento della grafia o il riconoscimento vocale.

Una delle reti ricorrenti più usate è senza dubbio la rete LSTM (Long short-term memory), una tipologia di rete utilizzata che consente di poter lavorare su sequenze di ingresso più lunghe. Sono in grado di gestire i problemi legati alla scomparsa del gradiente, una complicazione legata al processo della back-propagation durante il quale il valore dei gradienti (valori utilizzati per aggiornare i pesi di una rete neurale) diminuisce o aumenta mentre si propaga indietro nel tempo, il che rende le reti LSTM più adatte alla cattura delle dipendenze a lungo termine nei dati sequenziali. Vengono utilizzate per problemi tipici come: generazione di testo; riconoscimento vocale; traduzione automatica; previsione di serie storiche [25].

Di seguito in figura vi sono le strutture di un neurone LSTM e un RNN.

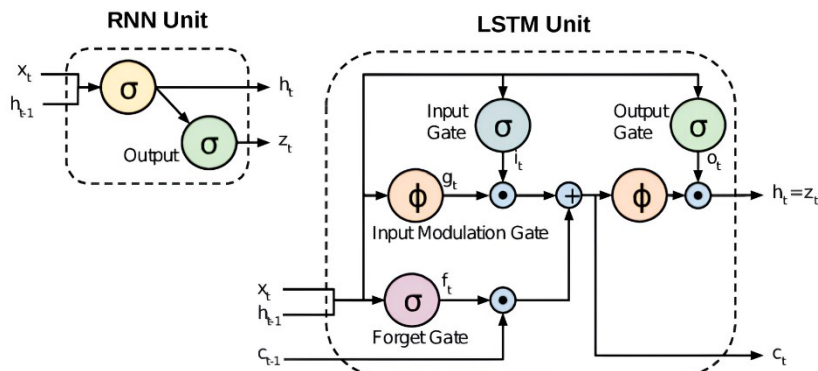


Figure 3.5: Confronto rete RNN classica e LSTM. Si veda [17]

Come si può notare dalla figura, il neurone LSTM è di gran lunga più complesso di un semplice neurone ricorrente; infatti, quest'ultimo controlla un'uscita supplementare h_t per lo stato attuale, un ingresso h_{t-1} per lo stato precedente e due diverse funzioni di attivazione per lo stato h e l'uscita z [17].

Il neurone LSTM è costituito internamente da diverse porte (*gate*), che gli consentono di scegliere cosa memorizzare o dimenticare durante la fase di training, combinare l'ingresso con lo stato interno e infine restituire l'uscita.

Nello specifico una LSTM è formata da:

- *Forget Gate*, determina quali informazioni verranno conservate o scartate. I dati vengono compressi tra 0 e 1 dalla funzione di attivazione; i valori vicini a zero vengono dimenticati, mentre quelli vicini a 1 vengono conservati, perché moltiplicare un elemento con uno 0 significa che quella

informazione non è rilevante, moltiplicarlo con 1 significa che l'informazione rappresentata è completamente rilevante.

- *Input Gate*, cerca di apprendere nuove informazioni dai suoi input;
- *Output Gate*, passerà le informazioni che sono state aggiunte o aggiornate dai gates precedenti, dal timestamp corrente a quello successivo.

Una LSTM oltre ad avere un *hidden state*, presente anche in una RNN, cioè la sezione short term memory; ha anche un *cell state*, la long term memory.

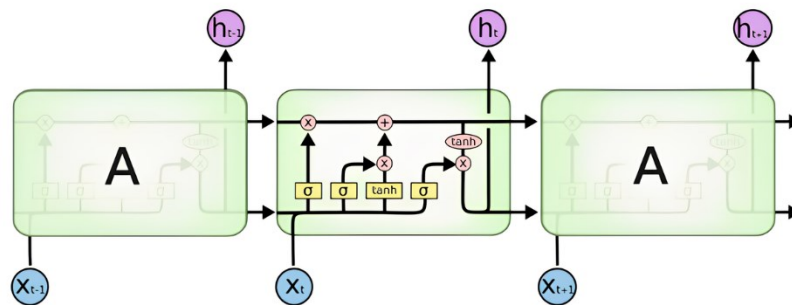


Figure 3.6: Anatomia rete LSTM [12]

Nella figura in alto si osserva l'architettura più dettagliata di una LSTM. Il cell state è rappresentato dalla freccia nella parte alta, mentre l'hidden state nella parte bassa.

Il *Forget gate* prende l'input dell'hidden state del timestamp precedente h_{t-1} e lo stato attuale (X_t) e decide quali informazioni tenere e quali dimenticare dal cell state corrente.

Ora è necessario decidere quali nuove informazioni aggiungere al cell state corrente, compito dell'*Input gate*. Esso prende in input gli stessi dati del precedente timestamp, così da far creare alla funzione *tanh* un vettore di nuovi valori; la funzione sigmoidea decide invece quali fra queste informazioni tenere.

Infine, con l' *Output gate* viene calcolato l'hidden state del timestamp corrente e la funzione sigmoidea determina quali informazioni del cell state, precedentemente aggiornato, lasciare passare in output e quali no [12].

Questo tipo di struttura permette di mantenere dati per un periodo di tempo molto lungo, con prestazioni migliori rispetto alle RNN tradizionali [9].

3.4 Settori d'applicazione delle ANN

L'ampia gamma di applicazioni delle reti neurali artificiali potrebbe essere la ragione del loro successo. Infatti, il loro paradigma di apprendimento consente di affrontare problemi di natura molto diversa e di trovare soluzioni con un impegno relativamente basso.

Con i recenti progressi, il ruolo delle reti neurali nelle applicazioni industriali è destinato ad aumentare in futuro, infatti secondo uno studio condotto da

MarketsandMarkets, il mercato globale delle reti neurali artificiali dovrebbe passare da un valore di 117 milioni di dollari nel 2019 a 296 milioni di dollari entro il 2024, spinto dal crescente interesse per l'intelligenza artificiale [2].

Qualche esempio pratico delle loro applicazioni: le reti neurali convoluzionali sono utilizzate per il riconoscimento facciale e l'elaborazione delle immagini. Per le previsioni del mercato azionario si tende a usare una *multilayer perceptron*. Le reti neurali artificiali vengono utilizzate per studiare i comportamenti degli utenti dei social media. I dati condivisi attraverso le conversazioni virtuali vengono raccolti e analizzati per vari tipi di analisi [4].

Capitolo 4 : Confronto tra Prophet e Reti LSTM

Nel corso del presente elaborato, sono stati esplorati due approcci distinti per la previsione di una serie storica, più precisamente del fatturato aziendale. È stata confrontata la precisione e l'efficacia della libreria Prophet e il metodo basato sulle Reti Neurali Artificiali, con un particolare approfondimento sulle reti LSTM.

In questo ultimo capitolo si studierà quanto precisamente è possibile “prevedere” il fatturato nel medio periodo di due ipotetiche aziende.

È importante notare che per ciascun metodo è stato adottato un approccio attento alla scelta del dataset di input al fine di ottenere risultati significativi. Per l'analisi con la libreria Prophet sono stati presi in considerazione dati giornalieri, fornendo una granularità temporale dettagliata. D'altra parte, nell'applicazione delle Reti Neurali LSTM è stato utilizzato un dataframe contenente dati espressi su base mensile.

4.1 Prophet

Prima di procedere con l'analisi, è necessario importare tutte le librerie rilevanti.

```
library(readxl)
library(scales)
library(patchwork)
library(ggplot2)
library(dplyr)
library(lubridate)
library(prophet)
```

Prima azione da compiere è caricare i dati su R tramite il comando `read_excel()` e rinomiamo la nuova tabella “*dati*”, essa è formata da due colonne, “*Data*” che rappresenta la serie temporale e “*Fatturato*” contenente i valori numerici del fatturato giornaliero.

```
# Caricamento del dataframe
dati <- read_excel("C:...dati.xlsx", sheet = "Daily")

# Visualizza i primi 5 valori e gli ultimi 5 valori
print(rbind(head(dati[, c('Data', 'Fatturato')], 5), tail(dati[,
c('Data', 'Fatturato')], 5)))

## # A tibble: 10 × 2
##   Data                Fatturato
##   <dtm>              <dbl>
## 1 2010-01-01 00:00:00    250.
## 2 2010-01-02 00:00:00    238.
## 3 2010-01-03 00:00:00    201.
## 4 2010-01-04 00:00:00    211.
## 5 2010-01-05 00:00:00    223.
```



```
##      ...      ...
## 6 2019-04-26 00:00:00 2560.
## 7 2019-04-27 00:00:00 2527.
## 8 2019-04-28 00:00:00 2546.
## 9 2019-04-29 00:00:00 2526.
## 10 2019-04-30 00:00:00 2562.
```

Si osserva che il periodo temporale del dataframe si estende dal 1° gennaio 2010 al 30 aprile 2019. Pertanto, di conseguenza le previsioni avranno inizio successivamente a quest'ultima data.

Tramite il comando di seguito è possibile osservare che non sono presenti dati mancanti; pertanto, non è richiesta alcuna azione correttiva in merito.

```
# controlliamo se ci sono NAs
any(is.na(dati))

## [1] FALSE
```

Attraverso l'ausilio del comando *ggplot()* è possibile creare un grafico che mostri la distribuzione del fatturato nel corso degli anni.

```
ggplot(dati, aes(x = Data, y = Fatturato)) +
  geom_line(color = "#0073C2", linewidth = 0.6) +
  labs(title = "Andamento del Fatturato",
       x = "",
       y = "") +
  theme_bw() +
  theme(plot.title = element_text(size = 12, hjust = 0.5),
        axis.title = element_text(size = 10)) +
  scale_y_continuous(labels = scales::dollar_format(prefix = "€"))
```

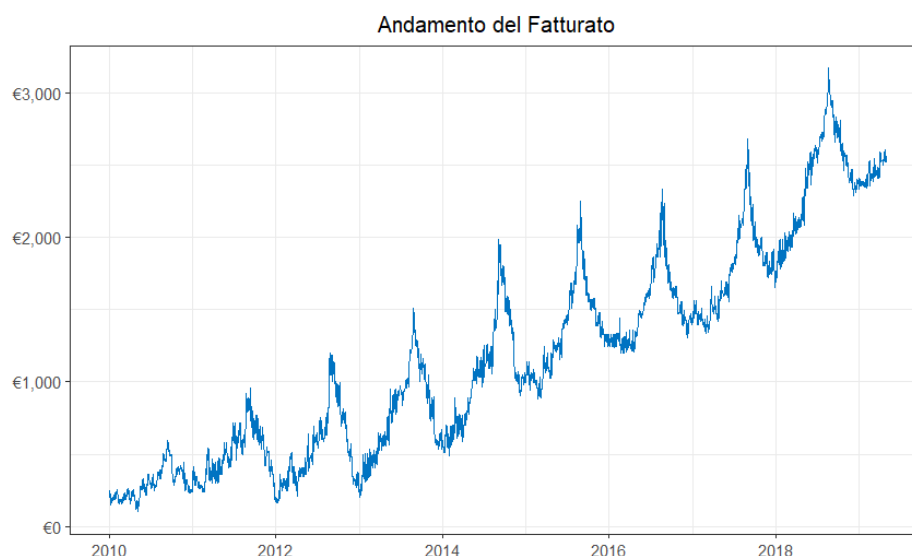


Figure 4.1: Andamento del Fatturato su R

Il grafico in alto fa notare l'andamento fortemente stagionale dei dati presi in considerazione, soprattutto nei mesi estivi vi è un picco del fatturato ogni anno.

Prima di iniziare con la vera e propria analisi è necessario creare una nuova tabella di nome “*prophet_data*” con colonne “*ds*” e “*y*” che rappresentano rispettivamente le colonne “*Data*” e “*Fatturato*” del dataframe “*dati*”.

Per rendere più attendibile la previsione è utile usare i valori logaritmici invece di quelli iniziali per la colonna che rappresenta il fatturato aziendale.

Dopodiché è possibile creare il *train* e il *test* set.

```
# Creazione nuovo Dataframe
prophet_data <- dati %>%
  rename(ds = Data,
         y = Fatturato) %>%
  mutate(y = log(y))

# Divisione in set di addestramento (80%) e test (20%)
train <- prophet_data[sample(nrow(prophet_data), round(0.8 * nrow(
  prophet_data))), ]

test <- prophet_data[setdiff(1:nrow(prophet_data), rownames(train
)), ]
```

Ora è il momento della creazione del vero e proprio modello previsionale della libreria Prophet con l’ausilio dei seguenti comandi:

- ***prophet()***: crea un modello Prophet utilizzando i dati di addestramento forniti.
- ***make_future_dataframe()***: crea un dataframe “*future*” che rappresenta il periodo futuro per il quale si desidera effettuare le previsioni; in questo caso vengono previsti 365 giorni in avanti con una frequenza giornaliera.
- ***predict()***: il comando utilizza il modello creato per effettuare le previsioni sul dataframe “*future*”, generando così la tabella “*forecast*” contenente le previsioni.

```
# Seed per la riproducibilità
set.seed(666)

# Creo il modello e il forecast di Prophet
model = prophet(train)
future = make_future_dataframe(model, periods = 365, freq = 'day')
forecast = predict(model, future)
```

La tabella “*forecast*” è formata da diverse colonne che rappresentano varie componenti della previsione, come quella additiva o moltiplicativa, l’andamento settimanale o annuale e i rispettivi limiti inferiori e superiori.

```
#
print(names(forecast))
```

```
## [1] "ds" "trend"
## [3] "additive_terms" "additive_terms_lower"
## [5] "additive_terms_upper" "weekly"
## [7] "weekly_lower" "weekly_upper"
## [9] "yearly" "yearly_lower"
## [11] "yearly_upper" "multiplicative_terms"
## [13] "multiplicative_terms_lower" "multiplicative_terms_upper"
## [15] "yhat_lower" "yhat_upper"
## [17] "trend_lower" "trend_upper"
## [19] "yhat"
```

Ai fini di questa analisi, però, verranno tenuti in considerazione solo “*ds*” e “*yhat*”, cioè le date e i valori predetti; è importante ricordare che i valori di “*yhat*” sono in scala logaritmica.

All’interno di “*forecast*” vi sono delle previsioni che corrispondono anche a date passate, questo perché il modello viene addestrato su dati storici e viene utilizzato per prevedere valori futuri sulla base di tali dati.

```
# Visualizza i primi 5 valori e gli ultimi 5 valori
print(rbind(head(forecast[, c('ds', 'yhat')], 5), tail(forecast[,
c('ds', 'yhat')], 5)))

##           ds      yhat
## 1  2010-01-01 5.190137
## 2  2010-01-02 5.179289
## 3  2010-01-03 5.177340
## 4  2010-01-04 5.184923
## 5  2010-01-05 5.182007
##      ...      ...
## 6  2020-04-24 8.158763
## 7  2020-04-25 8.151050
## 8  2020-04-26 8.152340
## 9  2020-04-27 8.163283
## 10 2020-04-28 8.163855
```

Tuttavia, l’accuratezza delle previsioni del modello diminuisce all’aumentare dell’orizzonte temporale. Se si volessero vedere solo i valori effettivi oltre il 30 aprile 2019 basterebbe creare un altro dataframe derivante da “*forecast*” che si fermi però ad una data più vicina.

```
# Creazione dataframe
valori_futuri <- forecast %>%
  filter(ds > ("2019-05-01") & ds <= ("2019-07-30")) %>%
  select(Data = ds, Fatturato = yhat) %>%
  mutate(Fatturato = exp(Fatturato),
         Data = as.Date(Data))

# Valori iniziali
head(valori_futuri)

##           Data Fatturato
## 1 2019-05-01 2643.825
```

```
## 2 2019-05-02 2668.686
## 3 2019-05-03 2694.200
## 4 2019-05-04 2692.570
## 5 2019-05-05 2715.060
## 6 2019-05-06 2763.642
```

Tramite il comando *plot()* si può visionare il grafico che mostra l'andamento dei dati futuri, già da questo grafico è possibile confermare l'ottimo addestramento del modello; infatti, si nota come l'andamento delle previsioni (in blu) segua quasi perfettamente quello dei dati reali.

Con *add_changepoints_to_plot()*, invece, vengono aggiunti i *changepoints* identificati automaticamente da Prophet, cioè dei momenti temporali in cui la serie mostra una deviazione significativa dalla tendenza.

```
# Grafico
plot(model, forecast) +
  add_changepoints_to_plot(model) +
  ylab("Fatturato") +
  xlab("Data") +
  theme_bw()
```

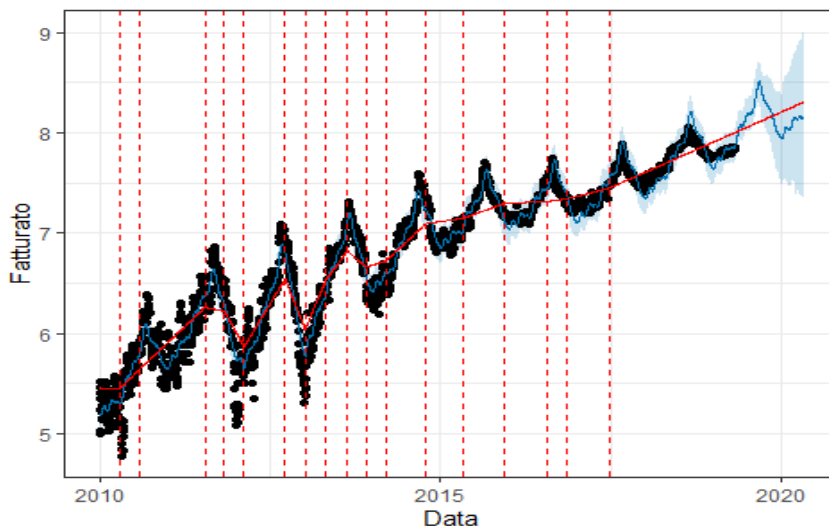


Figure 4.2: Andamento delle previsioni effettuate e Changepoints

Con il comando *prophet_plot_components()*, invece, si possono visualizzare le componenti principali delle previsioni; infatti, la funzione genera un insieme di immagini che mostra separatamente il trend: nel corso degli anni (prima sezione), durante i giorni della settimana (seconda sezione) e durante l'anno (terza sezione).

Grazie ad esse è possibile confermare che i dati presentano un trend positivo che rimane costante nel tempo dopo l'anno 2015. Da notare la forte componente stagionale, soprattutto nei mesi che vanno da luglio ad ottobre, con dei picchi del fatturato a settembre di ogni anno; mentre vi è un calo in inverno, particolarmente a gennaio e febbraio.

```
# Grafici
```

```
prophet_plot_components(model, forecast)
```

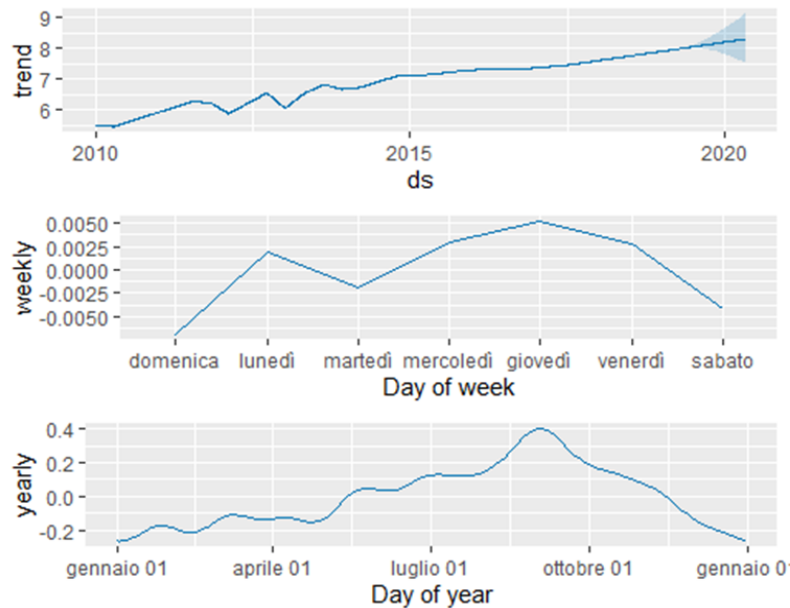


Figure 4.3: Componenti principali della previsione

Tutte le componenti visive create automaticamente dalla libreria sono utili soprattutto a creare un quadro generale del dataframe e della previsione, a favore di diverse interpretazioni utili.

Dopo aver esaminato le varie componenti che caratterizzano il modello, è possibile valutare la previsione tramite il calcolo delle metriche principali.

```
# Metriche
```

```
forecast_metric_data = forecast %>%
```

```
  as_tibble() %>%
```

```
  mutate(ds = as.Date(ds)) %>%
```

```
  filter(ds >= ymd("2017-06-19"))
```

```
RMSE = sqrt(mean((test$y - forecast_metric_data$yhat)^2))
```

```
MAPE = mean(abs((test$y - forecast_metric_data$yhat)/test$y))
```

```
#
```

```
cat("RMSE:", RMSE, "\n")
```

```
## RMSE: 0.3535926
```

```
cat("MAPE:", MAPE, "\n")
```

```
## MAPE: 0.03499191
```

I risultati in alto sono ottenuti tramite il confronto tra i dati della tabella “forecast” e del test set, quindi:

1. *RMSE*: Valore di 0.3535926. Suggerisce una deviazione moderata tra i valori previsti e quelli osservati.

2. *MAPE*: Valore di 0.0349919. In media, le previsioni sono sbagliate del 3.5% rispetto ai valori reali.

Come ultima metrica da calcolare rimane l' R^2 e per farlo si crea un modello di regressione lineare prendendo in considerazione i valori futuri di “*forecast*”, da notare che vengono utilizzati solo i valori predetti dal modello che vanno prima del 30 aprile 2019.

```
#  
pred <- forecast$yhat[1:2726]  
actual <- model$history$y  
  
# Grafico  
ggplot() +  
  geom_point(aes(actual,pred), color = 'darkblue', size = 1) +  
  labs(title = "Scatter Plot tra Valori Effettivi e Predetti") +  
  geom_smooth(method = "lm") +  
  geom_abline(intercept = 0, slope = 1, color = "red") +  
  theme_bw()
```

Il coefficiente R^2 , attestandosi a circa 0.9736, implica che il modello di
Scatter Plot tra Valori Effettivi e Predetti

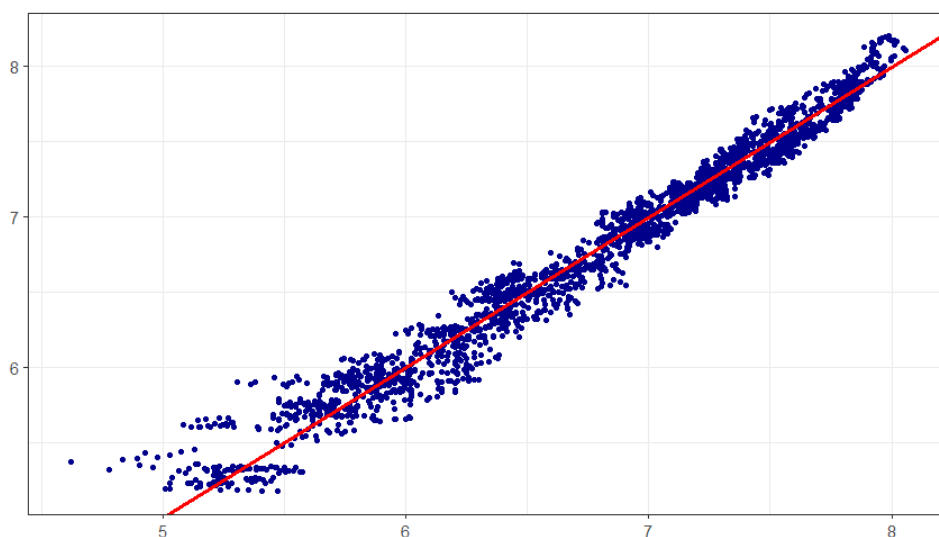


Figure 4.4: Scatter Plot su R

regressione elaborato tramite l'approccio di Prophet riesce a spiegare approssimativamente il 97.36% della variabilità osservata nei dati. Questo risultato mostra una significativa concordanza tra le osservazioni effettive e le stime fornite dal modello, confermando la capacità di quest'ultimo di catturare in maniera attendibile la struttura della serie temporale considerata.

```
# R^2  
R_squared <- summary(lm(pred~actual))$r.squared
```

```
cat("R-squared:", R_squared, "\n")
## R-squared: 0.9736358
```

In conclusione, l'analisi delle metriche di accuratezza del modello di previsione basato su Prophet presenta risultati incoraggianti. L'Errore Quadratico Medio (RMSE) e l'Errore Percentuale Assoluto Medio (MAPE) hanno valori molto bassi, il che significa in generale una buona performance. Mentre l'esame dell' R^2 evidenzia un eccellente adattamento del modello previsionale ai dati iniziali.

4.2 Reti LSTM

4.2.1 Breve Introduzione a Python e Analisi

Python è un linguaggio di programmazione creato nel 1991 da Guido van Rossum. Noto per la sua sintassi chiara e pulita, Python è utilizzato per lo sviluppo di applicazioni distribuite, scripting, computazione numerica e analisi dati. La sua natura open source consente il download e l'uso gratuito per chiunque e grazie alla vasta libreria built-in di Python offre molte funzionalità per diverse ambizioni [19].

Python è una scelta eccellente per l'analisi dati, grazie soprattutto alla sua facilità di apprendimento e alle sue ricche librerie come Pandas, Numpy e Matplotlib, che rendono più semplici le operazioni di manipolazione e visualizzazione dati [23].

Come per l'analisi con R, anche con Python è fondamentale importare i pacchetti necessari.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from keras.models import Sequential
from keras.layers import LSTM, Dense
```

Ora è possibile caricare il dataframe “df” tramite il comando *pd.read_excel()* e successivamente si imposta la colonna “Data” come indice. Da qui si nota come la serie temporale inizia il primo gennaio 1981 e si estende fino al 31 dicembre 2020. La tabella è composta da un totale di 480 righe, con i valori del fatturato aggregati su base mensile.

```
# Carichiamo il dataset e settiamo la colonna Data come indice
df = pd.read_excel("C:/.../dati.xlsx", sheet_name = "Monthly")
df = df.set_index('Data')
print(df)

##           Fatturato
## Data
```

```
## 1981-01-01    736.03
## 1981-02-01    760.42
## 1981-03-01    781.14
## 1981-04-01    771.03
## 1981-05-01    788.27
## ...          ...
## 2020-08-01   1135.80
## 2020-09-01   1180.50
## 2020-10-01   1147.70
## 2020-11-01   1094.30
## 2020-12-01   1138.50
##
## [480 rows x 1 columns]
```

Dopo aver confermato che il dataframe non presenta valori mancanti, è possibile visionare il grafico che mostra l'andamento dei dati.

```
# Controllo NAs
if df.isna().any().any():
    print("Ci sono dati mancanti nel DataFrame.")
else:
    print("Non ci sono dati mancanti nel DataFrame.")

## Non ci sono dati mancanti nel DataFrame.

# Grafico del DataFrame
df.plot(y = 'Fatturato', kind = 'line')
plt.title('Fatturato nel tempo')
plt.xlabel('Data')
plt.ylabel('Fatturato')
plt.show()
```

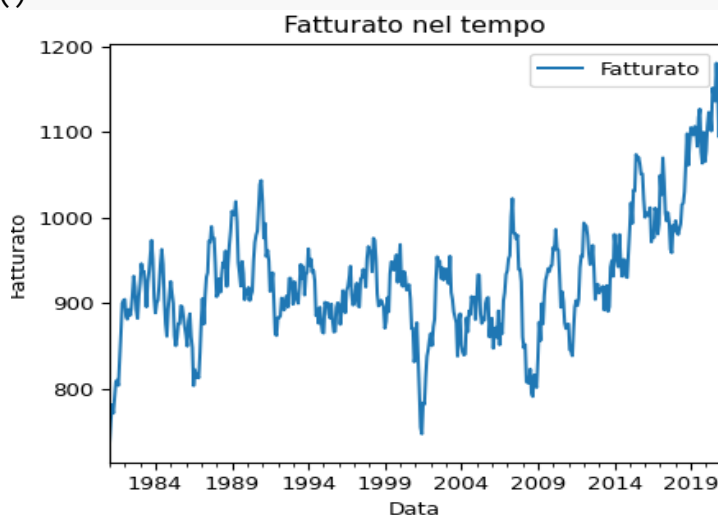


Figure 4.5: Andamento del Fatturato su Python

Infatti, si osserva che la dinamica non segue un trend lineare, bensì presenta variazioni significative del fatturato durante gli anni.

Il modello LSTM funzionerà prendendo in esame una sequenza di input per creare unica osservazione output. Si prende come esempio la sequenza:

[10, 20, 30, 40, 50, 60, 70, 80, 90]

Per facilitare tale apprendimento, è necessario trasformare la sequenza sopraindicata in una matrice formata da diversi campioni (*samples*) come righe, dai quali l'LSTM può estrarre informazioni significative.

Ogni campione è caratterizzato da passaggi temporali (*time steps*) che si distinguono in input e output. In questo caso i primi tre passaggi temporali vengono utilizzati come input mentre l'ultimo come output, che è stato appreso dal modello e impiegato per il campione successivo.

Di seguito la matrice di esempio in cui ogni campione è rappresentato dalle singole righe e i numeri sono i passaggi temporali; di conseguenza 10, 20 e 30 sono i valori di input con cui il modello verrà addestrato, mentre 40 sarà l'output:

```
[10 20 30] 40
[20 30 40] 50
[30 40 50] 60
[40 50 60] 70
[50 60 70] 80
[60 70 80] 90
```

La funzione `create_univariate_sequence()` di seguito implementa questo comportamento e dividerà una determinata sequenza univariata in più campioni in cui ciascuno di esso ha un numero specifico di passaggi temporali e l'output è un singolo passaggio temporale.

```
# Determinare la funzione che crea la sequenza univariata
def create_univariate_sequence(df_column, n_steps):
    sequence = df_column.tolist()
    X, y = list(), list()
    for i in range(len(sequence)):
        end_ix = i + n_steps
        if end_ix > len(sequence) - 1:
            break
        seq_x, seq_y = sequence[i:end_ix], sequence[end_ix]
        X.append(seq_x)
        y.append(seq_y)
    return np.array(X), np.array(y)
```

Dopo aver accertato il funzionamento del lavoro della rete LSTM è possibile creare la sequenza tramite la funzione `create_univariate_sequence()` determinando prima però il numero di passi temporali (*n_steps*) e standardizzare i dati affinché il modello possa leggere ed essere addestrato in maniera ottimale. Come valori della sequenza sono stati utilizzati quelli della colonna “Fatturato”.

```
# Scegliere il numero di passi temporali
n_steps = 3

# Creare la sequenza di input/output utilizzando "Fatturato"
```

```
X, y = create_univariate_sequence(df['Fatturato'], n_steps)
```

```
# Standardizzare i dati
```

```
scaler = MinMaxScaler(feature_range=(0, 1))
```

```
X = scaler.fit_transform(X)
```

```
y = scaler.fit_transform(y.reshape(-1, 1))
```

L'ultima fase prima della creazione della rete neurale necessita la riconfigurazione della struttura di “X”. Il modello richiede che il componente di input per i dati di addestramento abbia dimensioni pari a [campioni, timesteps, features] ed utilizzando una sequenza univariata il numero di features è uno, una sola variabile. Dal momento che la funzione *create_univariate_sequence()* descritta nella sezione precedente restituisce “X” con una forma di [campioni, timesteps], è necessario riconfigurare tale struttura per includere una dimensione aggiuntiva per le caratteristiche desiderate.

```
# Ridimensionamento
```

```
n_features = 1
```

```
X = X.reshape((X.shape[0], X.shape[1], n_features))
```

La fase di preparazione dei dati è conclusa. Infatti, dopo aver effettuato la suddivisione dei dati per l'insieme di addestramento e di test, si procede alla specifica del modello LSTM di tipo vanilla. Tale modello è caratterizzato da un unico strato nascosto e un singolo livello di output, impiegato per previsione.

```
# Dividere il set in Train e Test
```

```
train_size = int(len(X) * 0.8)
```

```
test_size = len(X) - train_size
```

```
X_train, X_test = X[0:train_size], X[train_size:len(X)]
```

```
y_train, y_test = y[0:train_size], y[train_size:len(y)]
```

```
# Seed per la riproducibilità
```

```
import random
```

```
random.seed(666)
```

```
# Definire il modello
```

```
model = Sequential()
```

```
model.add(LSTM(50, activation = 'relu', input_shape = (n_steps, n_features)))
```

```
model.add(Dense(1))
```

```
model.compile(optimizer = 'adam', loss = 'mse')
```

```
# Addestrare il modello
```

```
model.fit(X_train, y_train, epochs = 200, verbose = 0)
```

La procedura di inizializzazione del modello è condotta mediante l'impiego della funzione *Sequential()* che permette di creare modelli sequenziali strato dopo strato; mentre il comando *LSTM()*, il quale dispone di 50 unità di memoria, sfrutta una funzione di attivazione basata sull'unità lineare rettificata (ReLU), così definita:

$$f(x) = \max(0, x)$$

Successivamente, la funzione ***Dense()*** viene impiegata per aggiungere uno strato completamente connesso al modello, caratterizzato da un singolo neurone. La configurazione del modello per il processo di addestramento è attuata attraverso della funzione ***compile()***, che specifica l'ottimizzatore Adam e la funzione di perdita associata all'errore quadratico medio (MSE).

Infine, per l'addestramento verrà impiegata la funzione ***model.fit()***, con essa il modello utilizzerà l'algoritmo di ottimizzazione per ridurre la funzione di perdita tra le previsioni del modello e le etichette di addestramento. Questo processo viene ripetuto per il numero specificato di epoche, cioè quante volte il modello verrà addestrato, il quale sarà in grado di fare previsioni razionali sui dati di input forniti.

Dopo aver creato il modello ed averlo addestrato è possibile determinare le previsioni grazie alla funzione ***model.predict()*** e visualizzare graficamente sia i dati effettivi che quelli predetti al fine di valutarne le prestazioni.

```
# Predire sul set di test
y_pred = model.predict(X_test, verbose = 0)

# De-standardizzare le previsioni e i dati di test
y_pred_inv = scaler.inverse_transform(y_pred)
y_test_inv = scaler.inverse_transform(y_test)

# Plot delle previsioni e dei veri dati
plt.figure(figsize = (9, 6))
plt.plot(y_test_inv, label = 'Dati Veri')
plt.plot(y_pred_inv, label = 'Predizioni')
plt.title('Predizioni vs Dati Veri')
plt.xlabel('Osservazioni')
plt.ylabel('Fatturato')
plt.legend()
plt.show()
```

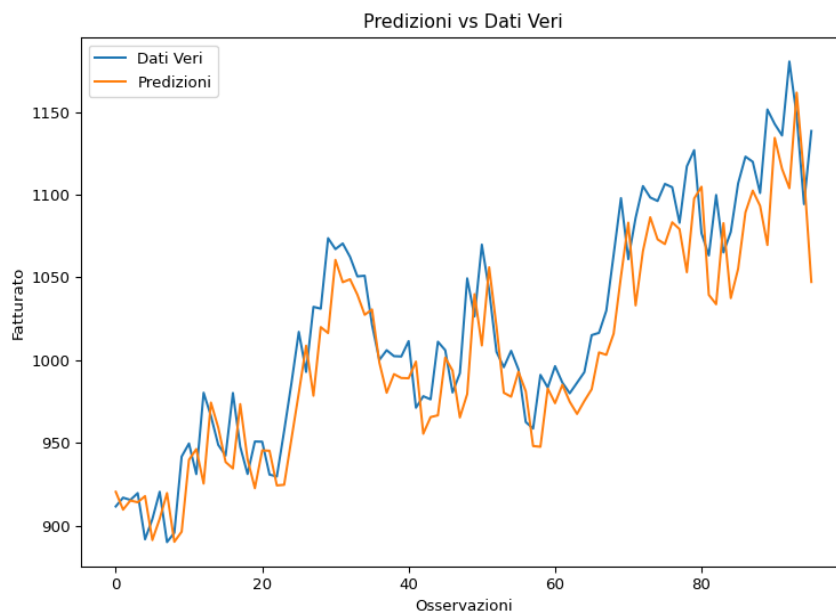


Figure 4.6: Confronto tra le previsione e i dati reali

Dall'analisi del grafico in alto emerge chiaramente che il modello è stato adeguatamente addestrato, manifestando una previsione estremamente positiva dei valori nel set di test.

Per prevedere il valore del mese di gennaio 2021, si utilizza nuovamente la funzione ***model.predict()***, la quale prende in considerazione l'oggetto "*last_sequence*", una matrice composta dagli ultimi tre valori del test set. Tale selezione è basata sulla considerazione dei tre passi temporali.

```
# Previsione del valore successivo
last_sequence = np.array(y_test[-n_steps:]).reshape((1, n_steps,
n_features))
next_value_standardized = model.predict(last_sequence, verbose=0)

# De-standardizzare le previsioni
next_value = scaler.inverse_transform(next_value_standardized.res
hape(-1, 1))
print("Valore futuro (De-standardizzato):", next_value[0, 0])

## Valore futuro (De-standardizzato): 1127.4319
```

Per valutare i risultati delle previsioni effettuate dalla rete neurale, è possibile implementare un modello di regressione lineare al fine di confrontare le previsioni generate con i dati effettivi presenti nel set di test, fornendo così una misura quantitativa della precisione della previsione.

```
# Creare lo scatter plot
plt.scatter(y_test_inv, y_pred_inv, marker='o', label='Dati')
plt.plot(x_line, y_line, color='red', label='Regressione Lineare')
plt.xlabel('y_test_inv')
plt.ylabel('y_pred_inv')
plt.title('Scatter Plot con Regressione Lineare')
plt.legend()
plt.show()
```

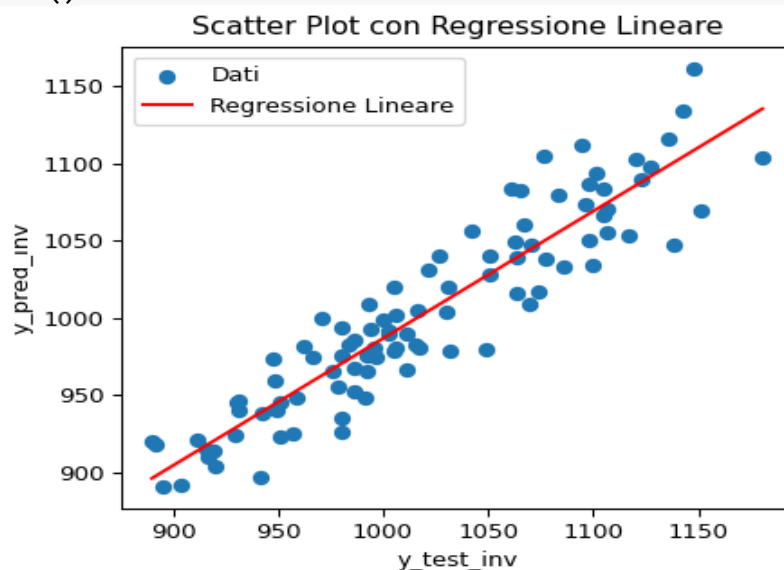


Figure 4.7: Scatter plot su Python

Di seguito invece vi sono i risultati delle metriche RMSE, MAPE, R^2 e MSE:

1. RMSE: 0.073. L'RMSE è una misura della deviazione media tra i valori effettivi e quelli predetti. In questo caso, il valore è molto basso, il che suggerisce che, in media, l'errore di previsione del modello è di circa 0.0726 unità.
2. MAPE: 8.853. Il MAPE è una misura percentuale dell'errore assoluto medio tra i valori effettivi e quelli predetti. In questo caso indica che, in media, le previsioni del modello hanno un errore percentuale assoluto medio dell'8.85%.
3. R^2 : 81.739. Misura quanto bene il modello di regressione spiega la variabilità nei dati. Il valore suggerisce che l'81.74% della variabilità nei dati può essere spiegata dal modello.

```
# Calcola l'RMSE (Root Mean Squared Error)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

# Calcola il MAPE (Mean Absolute Percentage Error)
mape = np.mean(np.abs((y_test - y_pred) / y_test)) * 100

# Calcola l'R quadro
r2 = r2_score(y_test, y_pred) * 100

# Stampa i risultati
print('RMSE:', rmse)

## RMSE: 0.073

print('MAPE:', mape)

## MAPE: 8.853

print('R^2:', r2)

## R^2: 81.749
```

In conclusione, le metriche suggeriscono che il modello di regressione lineare sta producendo risultati ragionevolmente buoni. Infatti, l'errore medio è relativamente basso, il MAPE indica un'accuratezza percentuale accettabile e l' R^2 suggerisce una buona spiegazione della variabilità nei dati.

4.3 Motivazioni della scelta di diversi Dataframe

Il confronto dei due metodi è caratterizzato dall'utilizzo di due insieme di dati con differenti cadenze temporali, giornaliera per il metodo Prophet e mensile per le reti LSTM. Questa scelta è motivata da una considerazione attenta delle caratteristiche proprio dei due metodi, tale decisione mira a fornire una valutazione adatta alle specifiche di ciascuna metodologia.

Tale approfondimento è finalizzato a chiarire la scelta iniziale delle frequenze temporali, evidenziando le motivazioni dietro la decisione di adottare un approccio specifico per ciascun metodo.

L'analisi procederà con un ulteriore confronto tra i risultati ottenuti mediante l'impiego del dataframe con cadenza mensile per il metodo con la libreria Prophet e il dataframe con cadenza giornaliera per le reti LSTM.

Per iniziare si caricano su R i dati con periodicità mensile utilizzati con le reti LSTM. Successivamente, si procede con la creazione di “*prophet_data*”, che rappresenta un dataframe con dati espressi in scala logaritmica e infine si effettua la suddivisione in set di addestramento (train set) e set di test (test set).

```
# Caricamento dati
data <- read_excel("C:../dati.xlsx", sheet = 'Monthly')

# Creazione nuovo Dataframe
prophet_data <- data %>%
  rename(ds = Data,
         y = Fatturato) %>%
  mutate(y = log(y))

# Divisione in set di addestramento (80%) e test (20%)
train <- prophet_data[sample(nrow(prophet_data), round(0.8 * nrow
(prophet_data))), ]
test <- prophet_data[setdiff(1:nrow(prophet_data), rownames(train
)), ]
```

In seguito, si creano il modello Prophet e i grafici che mostrano l'andamento delle previsioni e lo scatter plot che confronta i dati reali con quelli predetti.

```
# Modello Prophet e forecast
model = prophet(train)
future = make_future_dataframe(model, periods = 365, freq = 'day')
forecast = predict(model, future)

#
pred <- forecast$yhat[1:384]
actual <- model$history$y

# Grafici
a <- ggplot() +
  geom_line(data = subset(test, ds >= as.Date("2013-01-01")),
           aes(ds, y)) +
  geom_line(data = subset(forecast, ds >= as.Date("2013-01-01")),
           aes(ds, yhat), color = 'red') +
  xlab("Data") +
  ggtitle("Andamento Previsioni e Dati Reali ") +
  theme_bw()

#
b <- ggplot() +
  geom_point(aes(actual, pred), color = 'darkblue', size = 1) +
```

```
labs(title = "Scatter Plot Valori Reali e Predetti") +
geom_smooth(method = "lm") +
geom_abline(intercept = 0, slope = 1, color = "red") +
theme_bw()
```

a + b

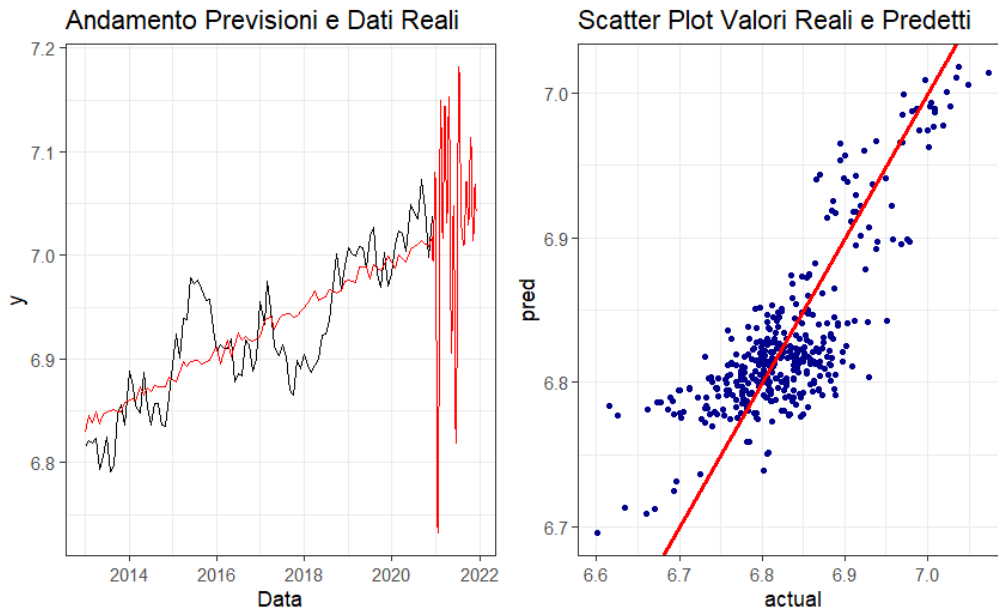


Figure 4.8: Andamento Previsioni e Scatter Plot su R

Dalla figura in alto emerge una dissonanza evidente tra le previsioni del modello e l'andamento effettivo dei dati. Tale discrepanza suggerisce la presenza di una sottostima nella capacità predittiva del modello, quasi sicuramente dovuta alla mancanza di stagionalità dei dati presi in esame. Anche il grafico a dispersione evidenzia delle discordanze rispetto a quello precedentemente esaminato, il quale risultava essere il prodotto di un'analisi basata su dati con frequenza giornaliera.

Nella fase successiva dell'analisi, ci si concentra sull'implementazione in Python delle reti LSTM utilizzando l'insieme di dati con frequenza giornaliera. Tuttavia, le reti LSTM producono un singolo output, pertanto predire un singolo giorno nel futuro potrebbe risultare limitato e poco significativo.

Al fine di ottenere risultati più pratici, è necessario aggregare i dati su base mensile. Questo approccio consente di ottenere previsioni più coerenti con le dinamiche aziendali, consentendo una prospettiva temporale più utile.

```
# Caricamento dati
df = pd.read_excel("C:/.../dati.xlsx", sheet_name = 'Daily')

# Creazione colonna 'Mese' e raggruppamento del fatturato
df['Data'] = pd.to_datetime(df['Data'])
df['Data'] = df['Data'].dt.to_period('M').dt.to_timestamp()
df = df.groupby('Data')['Fatturato'].sum().reset_index()
df = df.set_index('Data')
```

```
# Visualizzazione del dataframe
```

```
print(df)
```

```
##           Fatturato
## Data
## 2010-01-01    6186.76
## 2010-02-01    5254.48
## 2010-03-01    6615.69
## 2010-04-01    5887.21
## 2010-05-01    7054.09
## ...          ...
## 2018-12-01   73129.72
## 2019-01-01   73553.68
## 2019-02-01   68121.90
## 2019-03-01   75831.16
## 2019-04-01   76161.16
##
## [112 rows x 1 columns]
```

Organizzando i dati in base al periodo mensile, emerge immediatamente una significativa riduzione del numero di righe rispetto al dataframe precedentemente impiegato per le reti LSTM. Tale discrepanza potrebbe potenzialmente introdurre ambiguità nel modello.

Dopodiché è possibile proseguire con le fasi necessarie alla creazione del modello e al suo addestramento, similmente a quanto fatto in precedenza.

```
# Passi temporali e creazione sequenza di input e output
```

```
n_steps = 3
```

```
X, y = create_univariate_sequence(df['Fatturato'], n_steps)
```

```
# Standardizzare i dati
```

```
scaler = MinMaxScaler(feature_range=(0, 1))
```

```
X = scaler.fit_transform(X)
```

```
y = scaler.fit_transform(y.reshape(-1, 1))
```

```
# Ridimensionare
```

```
n_features = 1
```

```
X = X.reshape((X.shape[0], X.shape[1], n_features))
```

```
# Dividere il set in Train e Test
```

```
train_size = int(len(X) * 0.8)
```

```
test_size = len(X) - train_size
```

```
X_train, X_test = X[0:train_size], X[train_size:len(X)]
```

```
y_train, y_test = y[0:train_size], y[train_size:len(y)]
```

```
# Definire il modello
```

```
model = Sequential()
```

```
model.add(LSTM(50, activation = 'relu', input_shape = (n_steps, n_features)))
```

```
model.add(Dense(1))
```

```
model.compile(optimizer = 'adam', loss = 'mse')
```



```
# Addestrare il modello
model.fit(X_train, y_train, epochs = 200, verbose = 0)
```

Come ultimo step è possibile creare i grafici per valutare le performance delle previsioni effettuate dal modello.

```
# Creare il modello di regressione e adattarlo ai dati
m = LinearRegression()
m.fit(np.array(y_test_inv).reshape(-1, 1), y_pred_inv)
slope = m.coef_[0]
intercept = m.intercept_

# Creare una serie di valori x per la retta di regressione
x_line = np.linspace(min(y_test_inv), max(y_test_inv), 100)
y_line = slope * x_line + intercept

# Primo grafico
plt.subplot(1, 2, 1)
plt.plot(y_test_inv, label='Dati Veri')
plt.plot(y_pred_inv, label='Predizioni')
plt.title('Predizioni vs Dati Veri')
plt.xlabel('Osservazioni')
plt.ylabel('Fatturato')
plt.legend()

# Secondo grafico
plt.subplot(1, 2, 2)
plt.scatter(y_test_inv, y_pred_inv, marker='o', label='Dati')
plt.plot(x_line, y_line, color='red', label='Regressione Lineare')
plt.xlabel('y_test_inv')
plt.ylabel('y_pred_inv')
plt.title('Scatter Plot Regressione Lineare')
plt.legend()
plt.tight_layout()
plt.show()
```

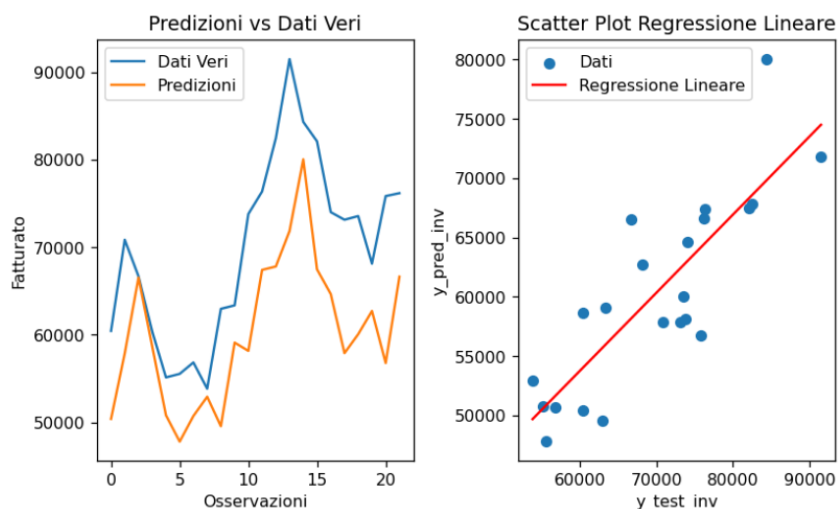


Figure 4.9: Andamento Previsioni e Scatter Plot su Python

In questa circostanza, è stato osservato che l'addestramento del modello ha mostrato prestazioni inferiori rispetto a quelle ottenute con reti LSTM in precedenza. La discrepanza è evidente nell'andamento delle previsioni, che non riflette correttamente l'evoluzione dei dati reali. L'analisi dello scatter plot fornisce un'ulteriore prova della scarsa efficacia del modello, sottolineando che la mancanza di dati sufficienti non ha portato a risultati soddisfacenti.

Conclusioni

Il confronto tra l'algoritmo Prophet su R e le reti LSTM su Python ha fornito risultati altamente promettenti, come dimostrato nel quarto capitolo. La validità di entrambi i metodi è stata dimostrata attraverso l'analisi di due dataset diversi, una scelta ponderata mirata a evidenziare le peculiarità di ciascun approccio.

Nel contesto della previsione del fatturato con dati a frequenza giornaliera e presenza di stagionalità, la libreria Prophet ha dimostrato di eccellere. La sua capacità di catturare tendenze ha prodotto risultati ottimali. Questo rende Prophet un'ottima scelta quando si tratta di previsioni che richiedono una sensibilità particolare alla dinamica temporale anche nel medio/lungo termine.

D'altro canto, l'uso di reti LSTM con un dataset a frequenza mensile ha rivelato altrettante prestazioni eccellenti che hanno portato un ottimo risultato per predire il mese subito successivo del dataframe.

La scelta di utilizzare due tabelle di dati distinte è stata motivata dalla natura intrinseca dei metodi analizzati. Prophet, essendo particolarmente adatto per dati con stagionalità, si è rivelato ideale per un contesto di dati giornalieri. Invece, le reti LSTM, producendo un singolo output, è stato più opportuno utilizzarle per fare una previsione mensile.

In sintesi, il confronto effettuato ha sottolineato l'importanza di adattare un metodo di previsione specifico in base alle caratteristiche dei dati disponibili.

Entrambi i metodi offrono vantaggi significativi e possono essere impiegati con successo nella previsione del fatturato aziendale, con la scelta del metodo più appropriato che dipende dalla natura e dalle esigenze specifiche dei dati disponibili.

Bibliografia

- [1] “Analisi Delle Serie Temporalì Con Facebook Prophet: Come Funziona E Come Usarlo.” n.d. ICHI.PRO. <https://ichi.pro/it/analisi-delle-serie-temporali-con-facebook-prophet-come-funziona-e-come-usarlo-265389504990778>.
- [2] “Artificial Neural Network Market by Solutions & Services - 2024 | MarketsandMarkets.” <https://www.marketsandmarkets.com/Market-Reports/artificial-neural-network-market-21937475.html>.
- [3] Coker, Frank (2014). *Pulse: Understanding the Vital Signs of Your Business* (1st ed.). Bellevue, WA: Ambient Light Publishing. pp. 30, 39, 42
- [4] “Cosa Sono Le Reti Neurali E Quali Le Loro Applicazioni.” 2022. UniverseIT. December 7, 2022. <https://universeit.blog/reti-neurali-cosa-sono/#:~:text=L>.
- [5] “Cos’è Il Machine Learning? | IBM.” <https://www.ibm.com/it-it/topics/machine-learning>.
- [6] De Paoli, Davide. 2020. “Reti Neurali Artificiali E Apprendimenti Basati Sulla Biofisica Dei Neuroni.” <https://amslaurea.unibo.it/22983/1/Tesi%20di%20laurea-Davide.DePaoli.pdf>.
- [7] Du, Ke-Lin, and M N S Swamy. 2014. *Neural Networks and Statistical Learning*. London Springer.
- [8] El Naqa, I., Murphy, M.J. (2015). What Is Machine Learning? In: El Naqa, I., Li, R., Murphy, M. (eds) *Machine Learning in Radiation Oncology*. Springer
- [9] Hochreiter, Sepp, and Jürgen Schmidhuber. 1997. “Long Short-Term Memory.” *Neural Computation* 9 (8): 1735–80.
- [10] “Home | TIOBE - the Software Quality Company.” n.d. [Www.tiobe.com](https://www.tiobe.com/). <https://www.tiobe.com/>.
- [11] Ihaka, Ross. (2022) "The R Project: A Brief History and Thoughts About the Future"
- [12] Incicco, Emanuele . 2022. “Identificazione Di Malware Dall’analisi Del Traffico DNS Attraverso Reti Neurali LSTM E Pre-Trained Embeddings.” Università Politecnica delle Marche. https://tesi.univpm.it/bitstream/20.500.12075/9288/1/Tesi_Magistrale_Emanuele_Incicco.pdf.

- [13] Letham, B., and Taylor S.J. 2017, Prophet: forecasting at scale, Facebook research, viewed 21 June 2018, <<https://research.fb.com/prophet-forecasting-at-scale>>.
- [14] Mandic, Danilo P, and Jonathon A Chambers. 2001. *Recurrent Neural Networks for Prediction*. John Wiley & Sons.
- [15] M. M. Hasan Shawon, S. Akter, M. K. Islam, S. Ahmed and M. M. Rahman, "Forecasting PV Panel Output Using Prophet Time Series Machine Learning Model," *2020 IEEE REGION 10 CONFERENCE (TENCON)*, Osaka, Japan, 2020, pp. 1141-1144
- [16] Ojha, Varun Kumar; Abraham, Ajith; Snášel, Václav (1 April 2017). "Metaheuristic design of feedforward neural networks: A review of two decades of research". *Engineering Applications of Artificial Intelligence*. 60: 97–116.
- [17] Pesci, Paolo . 2021. "Previsione Del Prezzo Delle Azioni Di S&P Con Reti Neurali LSTM E GRU." https://amslaurea.unibo.it/25208/1/pesci_tesi.pdf.
- [18] "Prophet: Forecasting at Scale - Meta Research." n.d. Meta Research. <https://research.facebook.com/blog/2017/02/prophet-forecasting-at-scale/>.
- [19] Python. 2019. "Python." Python.org. Python.org. May 29, 2019. <https://www.python.org/>.
- [20] "Reti Neurali Artificiali: Cosa Sono, Come Funzionano E Perché Vengono Costruite." n.d. Geopop. <https://www.geopop.it/reti-neurali-artificiali-cosa-sono-come-funzionano-e-perche-vengono-costruite/>.
- [21] Robson, Winston. 2020. "The Math of Prophet." Future Vision. November 12, 2020. <https://medium.com/future-vision/the-math-of-prophet-46864fa9c55a>.
- [22] Rosenblatt, F. (1958). "The perceptron: A probabilistic model for information storage and organization in the brain". *Psychological Review*. 65 (6): 386–408.
- [23] Sahoo, Kabita, et al. "Exploratory data analysis using Python." *International Journal of Innovative Technology and Exploring Engineering* 8.12 (2019): 4727-4735.
- [24] S. J. Taylor and B. Letham, "Forecasting at Scale", *The American Statistician*, vol. 72, no. 1, pp. 37-45, 2018.
- [25] Smagulova, Kamilya & James, A.. (2020). Overview of Long Short-Term Memory Neural Networks.

- [26] “To Predict or Not to Predict.” n.d. McCoy-Partners.com. <https://mccoy-partners.com/updates/to-predict-or-not-to-predict>.
- [27] Von, John. 1958. *The Computer and the Brain*. New Haven: Yale University.
- [28] Zell, Andreas. 1997. *Simulation Neuronaler Netze*. Bonn: Addison-Wesley.
- [29] 12.4 Neural Network Models | Forecasting: Principles and Practice (3rd Ed). n.d. Otexts.com. OTexts. Accessed February 1, 2024. <https://otexts.com/fpp3/nnetar.html>.